



User Guides

FIT3164
MDS8

Visual Perception from a Drone's Perspective

Group Member:

Emma Chong Yong Yong

Student ID: 31969763

Email: echo0039@student.monash.edu

Choo Jing Rou

Student ID: 31985246

Email: jcho0103@student.monash.edu

Fong Chun Kent

Student ID: 32189117

Email: cfon0015@student.monash.edu

Project Supervisor:

Professor Raphaël Phan

May 19th, 2023

Table of Contents

1. Introduction	3
2. Overview of Our Model	3
2.1 Description of Our Model	3
2.2 Main Models Used	3
2.3 Model Architecture	4
Phase 1: Dataset Preprocessing	4
Phase 2: Dataset Splitting	4
Phase 3: Model Training and Testing	4
Phase 4: Model Performance Evaluation	5
2.4 Limitations of Our Model	5
3. End User Guide	6
3.1 Basic Starting Command	6
3.2 Explanation of the Parameters	7
4. Technical Guide	8
4.1 First-Time Users	8
4.2 Environment Requirements	8
4.3 Installing the System	8
4.3.1 Setting Up the Environment.	8
4.3.2 Installing Pre-Trained Models	10
4.3.3 Preparing Your Own Dataset for Training	10
5. Common Errors and Solutions	12

1. Introduction

The purpose of this user guide report is to provide detailed instructions for both the end users and system administrators on the usage and setup of our human action recognition model. There are two main parts to this report: the End User Guide and the Technical Guide. The End User Guide is targeted for everyday users, and it will provide the detailed step-by-step instructions for the common and important tasks that the users, especially those interested in the topic, would encounter when using our model. On the other hand, the Technical Guide is targeted for system administrators responsible for setting up and configuring our model, and it will provide the detailed instructions on the installation and configuration processes to use our model.

2. Overview of Our Model

2.1 Description of Our Model

Our deep learning model is designed to analyse videos taken from an aerial view (drone's perspective), and attempts to accurately recognise the action(s) being performed by the individual featured in the videos. Currently, our model is trained to recognise 12 actions, which includes actions such as: walking (fb), running (fb), jogging (fb), punching, clapping, waving, stabbing, hitting with a stick, and hitting with a bottle.

*Note: fb = front/back view

2.2 Main Models Used

- COCO model
 - Used for feature extraction on our video datasets.
 - Extracts individual frames from the video to detect and localise the keypoints of the human body joints. The detected keypoints are then associated to form the complete human body poses, allowing for the tracking and capturing of human movements/actions over time.
- VGG_origin and mobilenet_thin
 - Both VGG_origin and mobilenet_then are OpenPose pre-trained deep learning models, used for pose estimation.
 - VGG_origin uses a deep architecture with multiple convolutional layers to extract features from images. Mobilenet_thin is a lightweight variant of the Mobilenet model that utilises depth-wise separable convolutions to reduce the computational complexity.
 - Although VGG_origin is able to achieve a higher accuracy, our model uses mobilenet_thin because of its faster inference speed, but at the cost of a lower but still reasonably high accuracy.

- Deep Neural Network (DNN)
 - A class of artificial neural networks with 4 layers between the input and output layers. It is capable of learning complex patterns and features from data.
- Multi-Layer Perceptron (MLP)
 - A fully connected class of feedforward artificial neural networks with hidden layer sizes of 20, 30 and 40. It is commonly used for classification predictions.
- Quadratic Discriminant Analysis (QDA)
 - A classification algorithm that models the probability distribution of each class using quadratic decision boundaries, and assumes that each class follows a Gaussian distribution. It is used in deep learning models to perform classification predictions.
- eXtreme Gradient Boosting (XGBoost)
 - An optimised gradient boosting algorithm that implements machine learning algorithms under the Gradient Boost framework with max depth of 8 and 500 estimators.

2.3 Model Architecture

Phase 1: Dataset Preprocessing

1. Feature Extraction:

The 18 body joints (keypoints) of the individual featured in the video are tracked. The x and y-coordinates and the confidence score for each keypoint detection are generated using the COCO model of OpenPose. Mobilenet_thin and VGG_origin are also used to retrieve the human skeletons.

2. Data normalisation:

After performing feature extraction on the videos, the x and y-coordinates are normalised by the frame height and width such that they are relative to the size of the video frame.

Phase 2: Dataset Splitting

Before training and testing our model, we split our dataset into two sets: Training Set and Testing Set. 90% of the dataset is used for training the model, and the remaining 10% is used for testing the model.

Phase 3: Model Training and Testing

The training dataset is used to train our four models, which are the DNN, MLP, QDA and XGBoost models. After training, the testing dataset is used to test our model.

Phase 4: Model Performance Evaluation

After the training and the testing phase, the model performance is evaluated by creating a confusion matrix and calculating the accuracy. Users also can train and test the mentioned four models again by changing the random state while splitting the dataset to view the performance of each model.

To view the video analysis visualisation, the user can retrieve the human skeleton using mobilenet_thin or VGG_origin, and choose one of the four models to recognize the human action(s).

2.4 Limitations of Our Model

Currently, our model has been trained and developed to recognise a limited number of 12 actions, which are listed in [Section 2.1](#). Besides that, our model has been trained and tested with video datasets that feature a single individual in each video frame. Hence, the performance of our model cannot be guaranteed beyond the mentioned limitations.

3. End User Guide

3.1 Basic Starting Command

1. Put your test videos into a directory and copy the directory path.

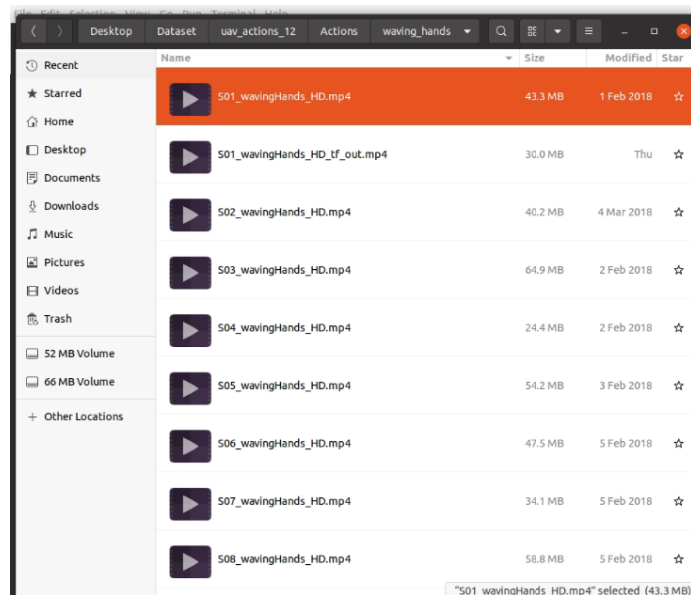


Figure 3.1 : Files in the directory

2. Open your terminal (command prompt), and change the directory to where the action recognition directory is located. Do this by copying the path of where the action recognition directory is located, then run the following command in the terminal:

```
cd <path>
```

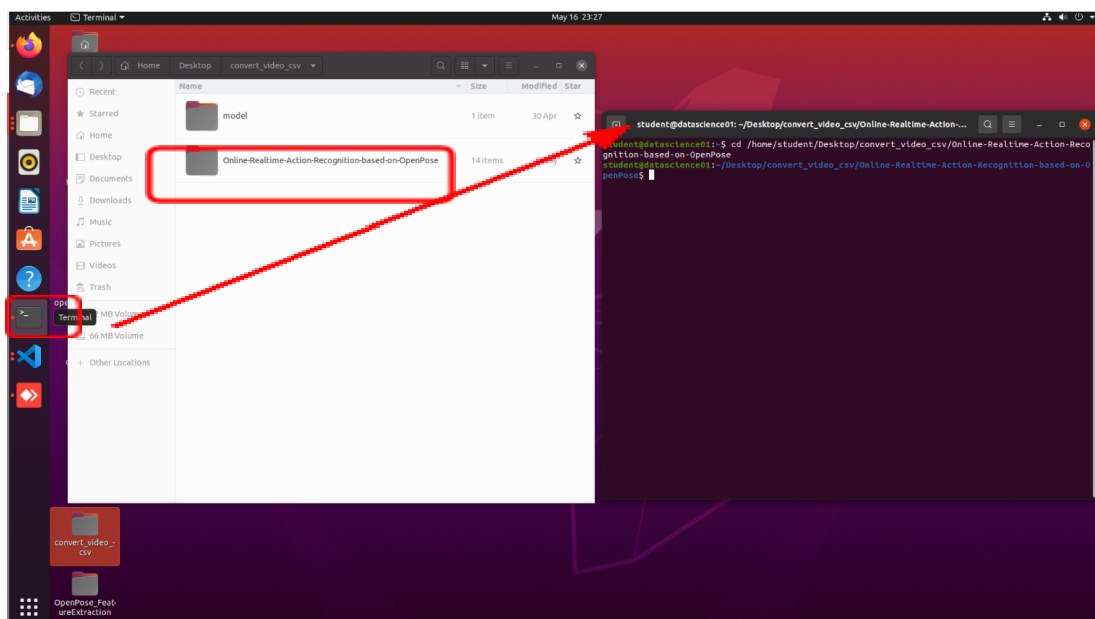


Figure 3.2 : Navigating to desired directory

3. In the same command line window from the previous section, run the following line:

```
$ python3 main.py --video <video_name> --model <model_name>
```

The explanations of the parameters are detailed in [Section 3.2](#). For this example, the video is called S01_wavingHands_HD.mp4, and we wish to identify its action performed by the person in the video. Also, the second argument we wish to use is the desired model for human action recognition, in this example we will use our XGBoost model, named action_recognition_XGBtestpp.h5. So, the appropriate command would be:

```
$ python3 main.py --video  
/home/student/Desktop/Dataset/uav_actions_12/Actions/waving_hands/S01_wavin  
gHands_HD.mp4 --model action_recognition_XGBtestpp.h5
```

```
student@datascience01:~/Desktop/convert_video_csv/Online-Realtime-Action-Recognition-based-on-OpenPose$ python3 main.py --video /home/student/D  
ownloads/S10_stabbing_toRight_HD.avi --model /home/student/Desktop/convert_video_csv/Online-Realtime-Action-Recognition-based-on-OpenPose/actio  
n_recognition_XGBtestpp.h5
```

Figure 3.3 : Example of command line with arguments

4. After running the steps stated above, the results will soon show up on your screen.

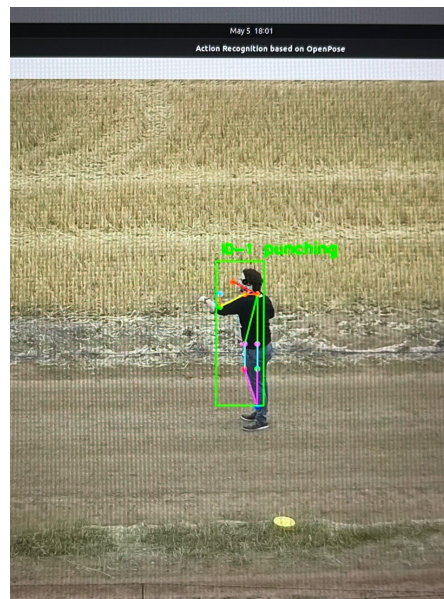


Figure 3.4:: Result of Action Recognition

3.2 Explanation of the Parameters

- **--video**
File path to the input video. The path should exist and the file extension should be included as well. For example, “punching.mp4”, instead of “punching”
- **--model**
File path to the saved trained model for action recognition. There is no default value for this parameter, therefore a model must be used. The file extension should be included.

4. Technical Guide

4.1 First-Time Users

- Ensure that your system meets the requirements stated in 4.2.
- Have a stable internet connection for a first time setup.
- There is no GUI for this current version. Everything is done via the command line.

4.2 Environment Requirements

- A recent Nvidia dedicated graphics card. (RTX2070 and above is recommended)
- A minimum of 16GB or more of high speed RAM (3200MHz or higher DDR4 RAM), 32GB or more is recommended.

4.3 Installing the System

4.3.1 Setting Up the Environment.

1. Install Python packages

- Run the following command lines to install the necessary packages:

```
$ pip3 install numpy  
$ pip3 install pandas
```

2. Install sklearn

- Run the following line in the command prompt to install:

```
$ pip3 install -U scikit-learn
```

3. Install tensorflow

- For CPU: run the following line in the command prompt:

```
$ pip3 install tensorflow
```

- For GPU: the steps are as follows

- a. Install the CUDA softwares, see: <https://www.tensorflow.org/install/gpu>, search for "Ubuntu 18.04 (CUDA 10)" or "Ubuntu 16.04 (CUDA 10)". Then copy and run the commands.

Step-by-step instructions

[Linux](#) [MacOS](#) [Windows Native](#) [Windows WSL2](#)

1. System requirements

- Ubuntu 16.04 or higher (64-bit)

TensorFlow only officially support Ubuntu. However, the following instructions may also work for other Linux distros.

★ **Note:** Starting with TensorFlow 2.10, Linux CPU-builds for Aarch64/ARM64 processors are built, maintained, tested and released by a third party: [AWS](#). Installing the [tensorflow](#) package on an ARM machine installs AWS's [tensorflow-cpu-aws](#) package. They are provided as-is. Tensorflow will use reasonable efforts to maintain the availability and integrity of this pip package. There may be delays if the third party fails to release the pip package. See [this blog post](#) for more information about this collaboration.

Figure 4.1. User Interface of Tensorflow Page for CUDA Installation

- b. Install tensorflow using the following command:

```
$ pip3 install tensorflow-gpu
```

4. Install tf-pose-estimation

- Download the openpose Github repository:

```
$ git clone https://github.com/gsethi2409/tf-pose-estimation
```

- Then, install all the required dependencies:

```
$ cd tf-pose-estimation
$ pip3 install -r requirements.txt
$ cd tf_pose/pafprocess
$ sudo apt install swig
$ swig -python -c++ pafprocess.i && python3 setup.py build_ext
--inplace
$ cd ../../
$ cd models/graph/cmu
$ bash download.sh
$ pip3 install slidingwindow
$ pip3 install opencv-python
$ pip3 install opencv-contrib-python
$ pip3 install simplejson
```

5. Install keras for action recognition

- Run the following line in the command prompt to install:

```
$ sudo -H pip3 install keras
```

4.3.2 Installing Pre-Trained Models

1. In [OpenPose](#), navigate to directory to /Pose/graph_models/VGG_origin.
2. Download the OpenPose VGG tf-model with the command line download.sh:

```
$ python3 ./download.sh
```

3. Repeat steps 1 and 2, with the directory /Pose/graph_models/mobilenet_thin.
4. We now have the pre-trained OpenPose models for pose recognition.

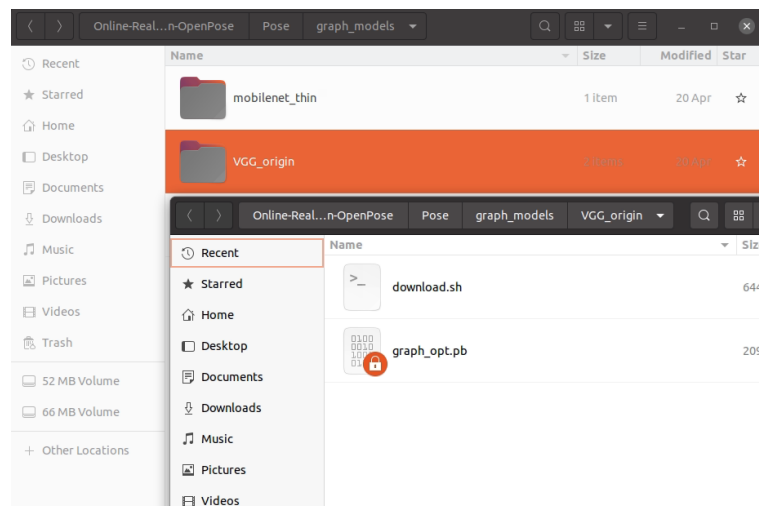


Figure 4.2. File graph_opt.pb for Pose Recognition

4.3.3 Preparing Your Own Dataset for Training

1. In the case of preparing your CSV file with your own folder of mp4 videos for model training, first compile the code at:
https://colab.research.google.com/drive/1rBkvPga_fr8QJi9oq9sRc_zJuqxT2dy6?usp=sharing#scrollTo=tgUKaDQZtF19

2. Change the runtime type to “GPU”

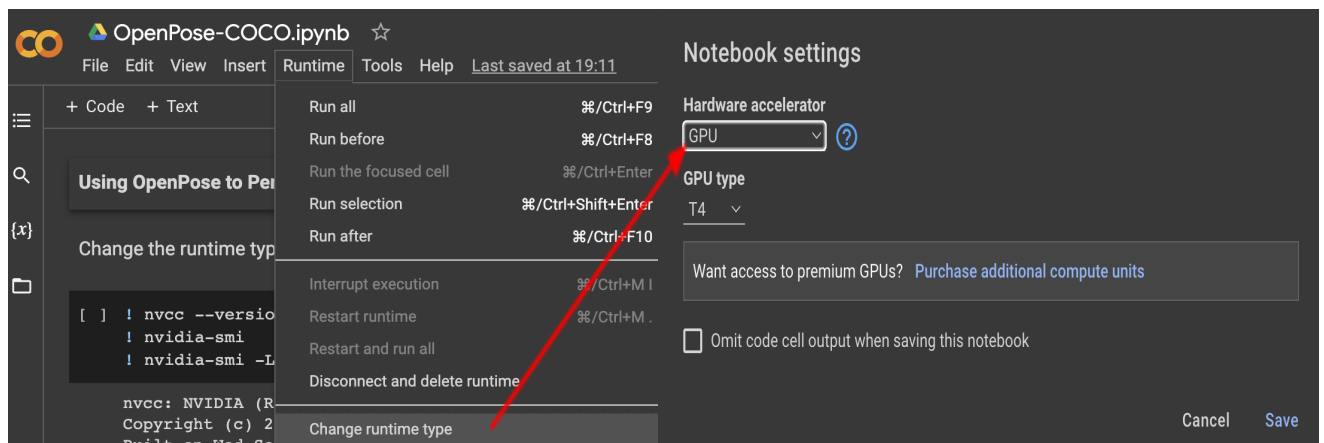


Figure 4.3. Configuration of Runtime Type

3. Upload your folder of videos onto Google Drive so that it is accessible in Google Colab.
4. Change the file paths accordingly in the Google Colab notebook.

```
[ ] input_directory = "/content/drive/MyDrive/HittingStick/hitting_stick"  
    input_data =  '/content/drive/MyDrive/HittingStick/hitting_stick/'  
    output_directory =  '/content/drive/MyDrive/HittingStick/hitting_stick_json/'
```

Figure 4.4. Configuration of File Path

5. Common Errors and Solutions

- OSERROR, file signature not found, although trained models file exist. (hd5.format)

```
OSError: Unable to open file (File signature not found)
```

- This is common as the file might be corrupted. Try checking the file with the HDF5 utility h5debug (available on command line if you've installed the hdf5 lib on your OS, check with `dpkg -s libhdf5-dev` on Linux).
- Another common fix would be to download the file again, or simply place the file under the same folder as the working directory (folder of the codebase)

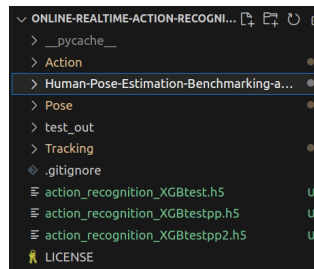


Figure 5.1 : Solution to OSError

- Tf-TRT warning, could not find TensorRT.

```
To enable the following instructions: AVX2 AVX512F FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.  
2023-05-19 17:30:16.849040: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT
```

Figure 5.2 : Tf-TRT warning

- This could be due to faulty installation of Tensorflow, or TensorRT not installed. Do run the command below to fix :

```
python3 -m pip install --upgrade tensorrt
```

- Could not find cuda drivers on your machine, GPU will not be used.

```
2023-05-19 17:30:16.278938: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.  
2023-05-19 17:30:16.324521: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.
```

Figure 5.3 : Could not find CUDA drivers warning

- The CUDA drivers may not be installed correctly, try the steps below :

- Check CUDA version :

```
nvcc --version
```

- Check if the connection with the CUDA libraries is correct:

```
echo $LD_LIBRARY_PATH
```

- Install TensorFlowRT through Nvidia.

```
pip install nvidia-pyindex  
pip install nvidia-tensorrt
```